

ASSIGNMENT 02 - FSSTAT

1. ANALISI DEL PROBLEMA

Il progetto richiede lo sviluppo di una libreria asincrona in Java, chiamata FSStatLib, che sia in grado di raccogliere statistiche sulla dimensione dei file trovati in una determinata directory del file system. La directory deve essere scansionata ricorsivamente e la libreria ha il compito di calcolare il numero totale di file e, dato un numero di intervalli dimensionali, la distribuzione dei file in ciascuno di questi. Ad esempio, la nostra implementazione di default presenta 10 intervalli da 10 kB ciascuno: in questo caso, il programma deve calcolare quanti file sono presenti nell'intervallo 0-10 kB, 10-20 kB, fino ad arrivare a 90-100 kB, e infine determinare gli outliers(100 kB+).

Risolvere questo tipo di problema con un approccio sincrono, comporterebbe come difetto una grave lentezza del programma. Quindi, l'approccio più efficace da utilizzare è far lavorare più entità contemporaneamente. Tuttavia, questo metodo introduce diverse sfide:

- Il problema della Race Condition, situazione che accade quando diversi thread o processi provano a modificare una risorsa condivisa nello stesso momento, generando risultati imprevedibili. Questa sfida si presenta anche in FSStatlib, dato che tutte le entità che esplorano le cartelle annotano i risultati della ricerca in un unico oggetto condiviso, chiamato Report. È stato necessario quindi progettare un sistema che resolvesse questa condizione rallentando il meno possibile l'esecuzione tramite la parallelizzazione
- Il problema dell'esaurimento delle risorse: una determinata directory può contenere tantissime sottocartelle diverse, e un approccio basato sul creare un nuovo Thread per ogni cartella scoperta porterebbe all'esaurimento delle risorse e al crash del sistema. Per risolvere questo problema abbiamo implementato tre diverse strategie (Virtual Threads, RxJava, Event-Loop), dove ognuna di esse sfrutta meccaniche di parallelizzazione e paradigmi diversi.

2. ARCHITETTURA E DESIGN DEL SISTEMA

Per risolvere il problema dell'esplorazione concorrente, abbiamo sviluppato tre approcci che lavorano in maniera differente, ma che presentano un metodo

getFSReport che deve svolgere lo stesso tipo di operazioni. Tale metodo asincrono calcola e restituisce un oggetto Report contenente:

- il numero totale di file appartenenti alla directory e sottodirectory contenute in quella iniziale
- la distribuzione delle dimensioni dei file: data una dimensione massima MaxFS e un numero di fasce dimensionali NB che dividono l'intervallo [0, MaxFS], il metodo calcola per ciascuna fascia il numero di file con dimensione rientrante in essa, nonché il numero di file con dimensione superiore a MaxFS (ottenendo, in questo modo, NB + 1 intervalli dimensionali). La directory, la dimensione massima e il numero di fasce dimensionali sono parametri del metodo.

L'oggetto Report è una classe che calcola le statistiche, trovando il numero totale di file e spostandoli nelle rispettive fasce. La struttura dati che abbiamo utilizzato per smistarli è una TreeMap. L'aggiunta dei file avviene in modo atomico, in quanto il metodo addFile inserisce il singolo file nella banda corrispondente alla sua dimensione. I metodi successivi permettono di ottenere in modo pratico informazioni sul report corrente. Questa classe viene usata e istanziata in tutte le varianti della libreria, rivelandosi una soluzione pratica ed estendibile.

NOTA: per ciascuno dei seguenti paradigmi, è stato effettuato un test di performance usando lo stesso hardware e eseguendo la ricerca sulla stessa cartella. Utilizzando la funzione nella versione interattiva della libreria viene tenuta traccia del tempo trascorso dall'inizio alla fine dell'analisi, dando un insight di quale potrebbe essere il paradigma migliore per questo tipo di problema.

2.1 Vert.x

L'approccio con Vert.x rappresenta una versione puramente asincrona e non bloccante del progetto, essendo basato sul paradigma dell'Event-Loop. Il codice delega la lettura del file system ai thread di background, e vengono usate Promise per "attendere" i risultati e aggiornare i contatori in modo sicuro. L'applicazione è strutturata attorno a un componente principale, FSStatEventLoop che estende VerticleBase, e alla libreria FSStatLib.

Il codice utilizza le API asincrone fornite da Vert.x. Infatti, viene eseguita una lettura asincrona (fs.readDir(dir)). Il risultato viene gestito all'interno di una callback (onComplete), all'interno della quale iteriamo i file e, per ciascuno di essi, estraiamo le proprietà per verificare se l'entità è una sottocartella o un file. Nel primo caso viene richiamata ricorsivamente la funzione getFSRecursiveReport e generata una nuova Promise. Nel secondo caso, invece, viene chiamata la funzione addFile della classe Report.

La gestione dell'albero delle directory richiede un meccanismo per capire quando l'esplorazione di un intero ramo può considerarsi conclusa. Abbiamo risolto questo problema usando il costrutto `Future.all(futures)`, in modo che la Promise associata ad una directory padre venga risolta solo quando tutte le Promise figlie hanno completato la loro esecuzione.

Infine, l'implementazione non necessita di bloccare o proteggere i dati (per evitare le Race Conditions). Infatti, la chiamata alla funzione `addFile` viene effettuata in totale sicurezza senza l'utilizzo di blocchi `synchronized` o strumenti come `Lock`.

Tutto questo è possibile grazie all'Event-Loop di Vert.x e alla chiamata alla funzione `vertx.deployVerticle`, in cui il framework prende il `FSStatEventLoop` e lo affida in modo esclusivo ad un solo thread. È l'event loop, quindi, che effettua chiamate asincrone (`fs.readDir`, `fs.props`, ...) in cui delega ai thread in background dei lavori più pesanti.

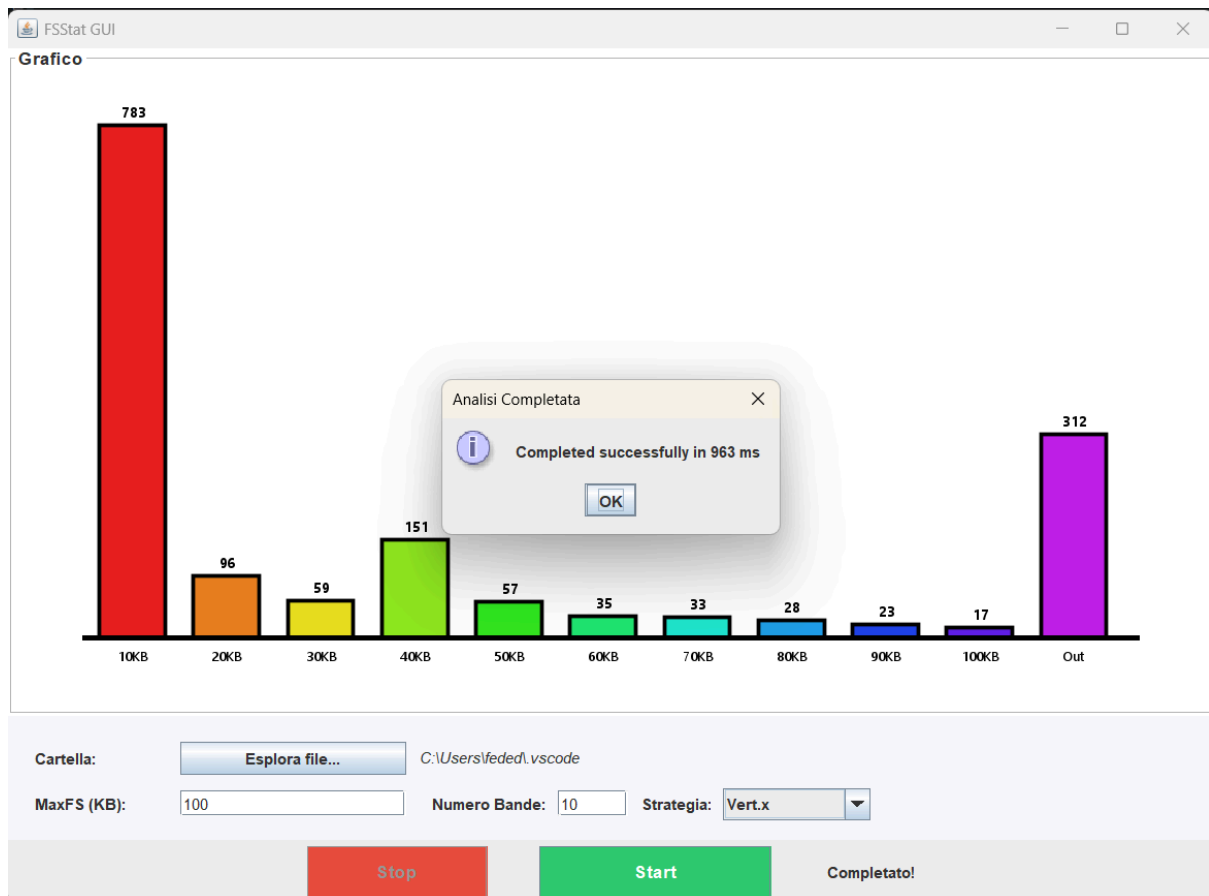
In questo modo, il problema della Race Condition viene eliminato in partenza: non ci sono mai due thread che modificano lo stesso oggetto nello stesso momento.

L'unica cosa da gestire era la corretta coordinazione tra le risposte asincrone (Promise).

Versione interattiva Vert.x

Per integrare questa architettura con l'interfaccia grafica, abbiamo adattato la classe principale in modo tale da non farle restituire un risultato finale al termine dell'operazione, ma per comunicare in modo asincrono tramite il pattern Observer. Infatti, passando un'interfaccia pubblica `StatListener` al metodo `startAnalysis` (diventato il nuovo `getFSReport` delle classi con interfaccia grafica), la libreria comunica gli stati di avanzamento, gli errori e la fine dell'esecuzione, in modo tale poi che il risultato venga mostrato a video nel pannello.

Per quanto riguarda la gestione del tasto "stop", abbiamo introdotto una variabile booleana `isStopped` che tiene monitorata l'effettiva messa in esecuzione del programma. Nella GUI, sono presenti i pulsanti `start` e `stop`, a cui sono associati dei listener che rispettivamente attivano i metodi `"startAnalysis"` e `"stop"`. Nel primo viene lanciata la prima chiamata al metodo ricorsivo `getFSRecursiveReport`, con i rispettivi comportamenti da tenere in caso di interruzione, successo e errore. Durante il metodo ricorsivo della scansione del file system, vengono eseguiti continui controlli per capire se è stata chiamata la fine dell'esecuzione. In tal caso, viene risolta immediatamente la promise e la ricorsione si interrompe.



2.2 RxJava

La seconda versione adottata è quella basata su RxJava, libreria che implementa le Reactive Extensions per la Java Virtual Machine. Il file system viene modellato come uno stream (o flusso di dati) e man mano che vengono esplorate le cartelle, vengono emessi eventi in tempo reale, che corrispondono alle dimensioni dei file in byte.

Per creare il flusso abbiamo utilizzato un Observable. La scelta di tale approccio a discapito del Flowable. Infatti, il Flowable introduce la gestione della Backpressure (necessaria quando i dati prodotti sono molto più veloci dei dati consumati). Come scelta progettuale, avendo deciso di non forzare la parallelizzazione ma di mantenere il flusso su un unico thread, l'emissione del dato e l'inserimento di tale dato nel Report avvengono in modo sequenziale. Quindi, il rischio di avere la memoria satura è azzerato, in quanto il produttore aspetta che il file venga letto e inserito nella rispettiva fascia prima di leggere il dato successivo. La Backpressure diventa superflua.

Nel nostro progetto, i flussi di dati generati sono di tipo Cold Stream: il flusso è definito Cold perché l'esplorazione del file system non parte da sola, ma attende che venga invocato un determinato metodo di sottoscrizione. Per fare in modo che non

venga bloccato il thread principale, abbiamo invocato il metodo `.subscribeOn(Schedulers.io\(\))`, per fare in modo di delegare l'intera esecuzione del codice a un thread di background gestito da RxJava, ottimizzato specificamente per operazioni I/O Bound.

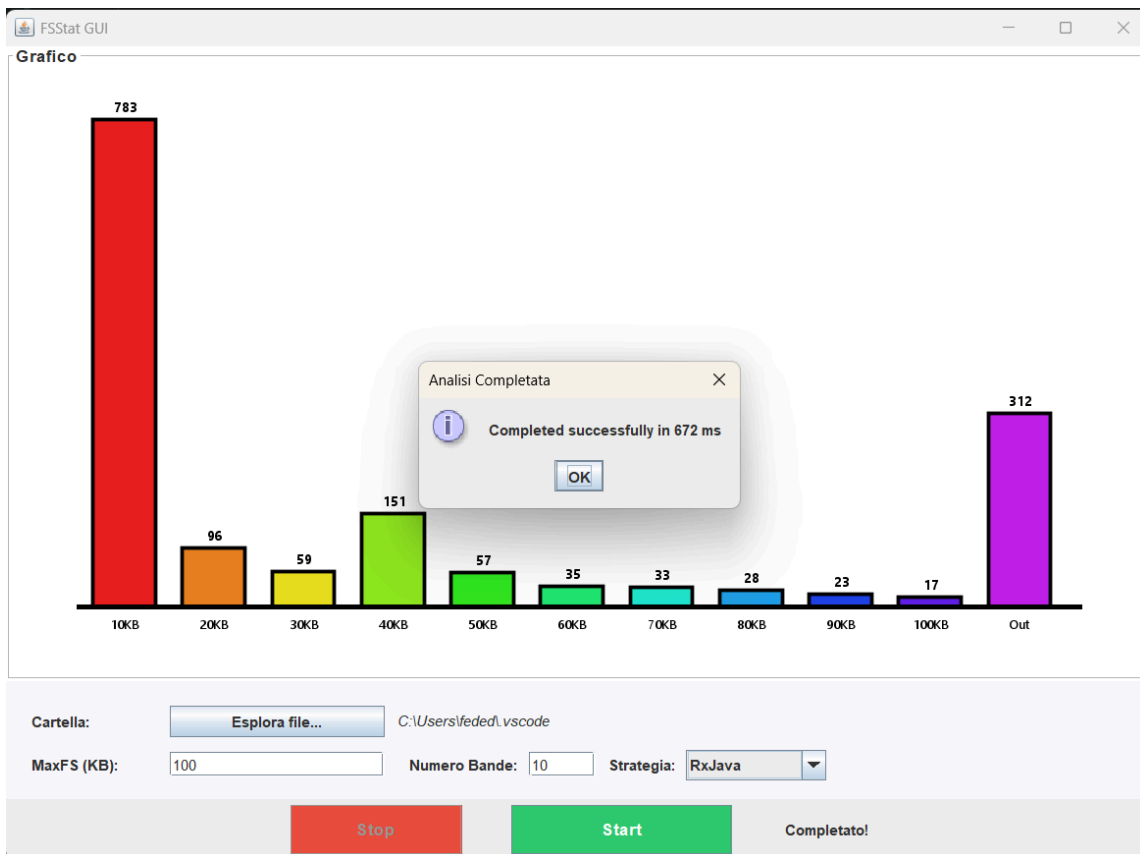
Infine, abbiamo usato il metodo `.blockingSubscribe(report::addFile)`, che inizializza l'analisi asincrona delle directory e associa un Consumer che intercetta ogni valore emesso (un valore Long, che rappresenta la dimensione in byte di ciascun file) e lo passa come argomento al metodo `addFile`. Essendo "blocking", il metodo principale interrompe la propria esecuzione rimanendo in attesa del completamento del flusso. Tramite il comando `.onComplete`, l'esecuzione riparte e viene restituito l'oggetto Report completo di statistiche.

All'interno del codice ricorsivo `recursiveFileSearch`, abbiamo inserito dei controlli (`if (emitter.isDisposed()) return;`) per fare in modo che l'esplorazione si interrompa nel momento in cui si verifica un errore critico. Altrimenti, il thread in background continuerebbe a ispezionare le directory anche dopo la chiusura del flusso.

Versione interattiva di RxJava

Nella versione interattiva, la modifica più importante riguarda la sottoscrizione al flusso. Infatti, abbiamo sostituito l'operatore `.blockingSubscribe()` con un `.subscribe()`: il primo blocca il thread chiamante fino a che non è terminata la scansione, mentre il secondo permette al thread principale di tornare a monitorare eventuali pulsanti premuti dall'utente.

Anche in questo caso è stata utilizzata l'interfaccia `StatListener` per mantenere aggiornato l'utente sullo stato di esecuzione del programma. La variabile booleana `isStopped` è stata introdotta anche in questa versione. In particolare, il controllo di tale variabile viene effettuato sia all'inizio del flusso di codice (prima di notificare la GUI), sia durante la chiamata ricorsiva.



2.3 Virtual Threads

Nella versione basata sui Virtual Threads, abbiamo creato un sistema che genera dinamicamente un nuovo Virtual Thread per ogni task sottomesso, permettendo di creare migliaia di thread leggeri senza esaurire la memoria o i thread del sistema operativo. Infatti, lo stack di un Virtual Thread viene gestito direttamente dalla Java Virtual Machine dentro l'Heap (la memoria gestita dal Garbage Collector).

La scansione dei file avviene nel metodo `recursiveFileSearch`. Quando il codice trova una sottocartella, non la esplora in modo sequenziale, ma delega la chiamata ricorsiva all'Executor tramite il comando `executor.submit()`. Ogni sottomissione restituisce un oggetto `Future` che viene salvato in una lista. Tale lista viene iterata alla fine dell'analisi richiamando `task.get()` per garantire che l'esecuzione principale non termini prima del dovuto.

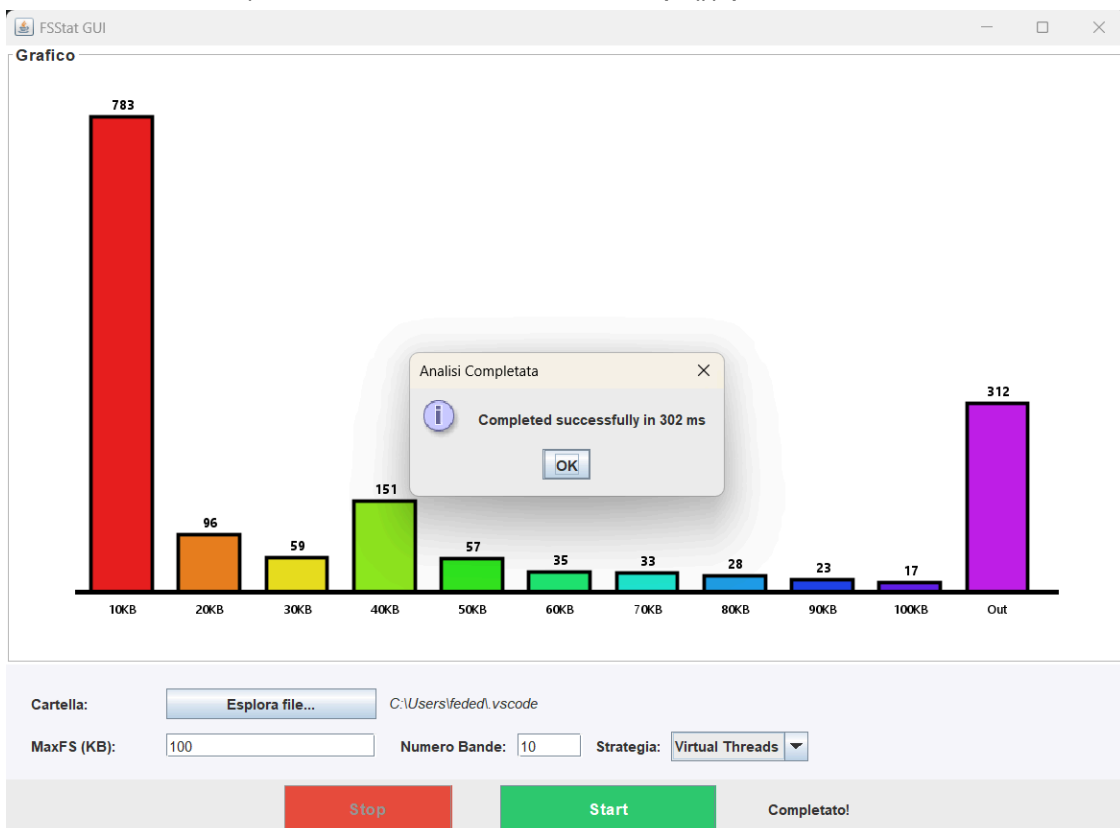
L'oggetto `Report` è condiviso tra tutti i Virtual Threads attivi. Quando viene trovato un file ed eseguita l'istruzione `addFile`, si entra in una sezione critica che può potenzialmente comportare una `Race Condition`. Per risolvere il problema e garantire la mutua esclusione, abbiamo utilizzato un `ReentrantLock`. Per evitare la situazione in cui due thread trovano contemporaneamente un file e invocano nello stesso

istante il metodo `addFile`, ogni virtual thread che vuole richiamare quel metodo prende prima un lucchetto per poter aggiornare i dati in totale sicurezza. La scelta del `ReentrantLock` ha prevalso sui blocchi `synchronized` perché questi ultimi non sono efficienti per le prestazioni. Usando `synchronized`, infatti, fuoriesce l'effetto "pinning", in cui un virtual thread rimane incollato al thread nativo del sistema operativo (il carrier thread) e non può essere staccato da esso. Questo vanifica i benefici di scalabilità del paradigma. Il `ReentrantLock`, al contrario, permette al Virtual Thread in attesa di liberare il thread di base per altre operazioni.

Versione interattiva di Virtual Threads

In questa versione, l'architettura è stata modificata in modo da renderla reattiva verso l'esterno. Il metodo `startAnalysis` usa un `executor.submit()` per avviare l'esecuzione in background, restituendo il controllo anziché bloccare il thread principale con `Future.get()`. In questo modo garantiamo che l'interfaccia non si blocchi durante i calcoli, con l'aggiornamento dei risultati gestito tramite i metodi di `StatListener`.

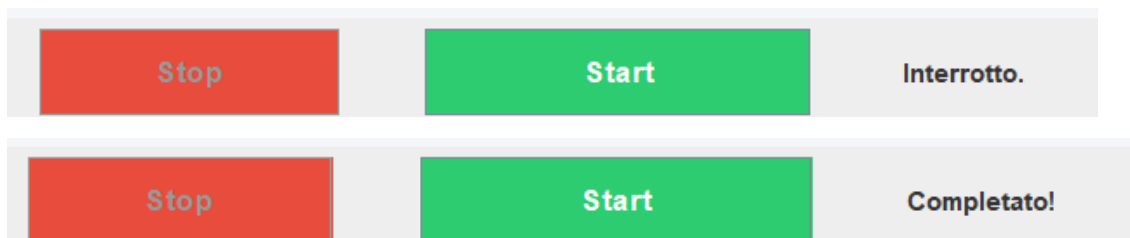
Quando viene richiesto lo stop, la variabile `isStopped` viene impostata a `true` e invoca `executor.shutdownNow()`, che invia un segnale di interrupt a tutti i virtual threads in attesa. Lo abbiamo gestito al meglio andando a catturare esplicitamente l'eccezione `InterruptedException` durante l'attesa del termine delle sottocartelle (`task.get()`). Tale eccezione non viene soppressa, ma ripristina lo stato di interruzione del thread corrente (`Thread.currentThread().interrupt()`) prima di uscire.



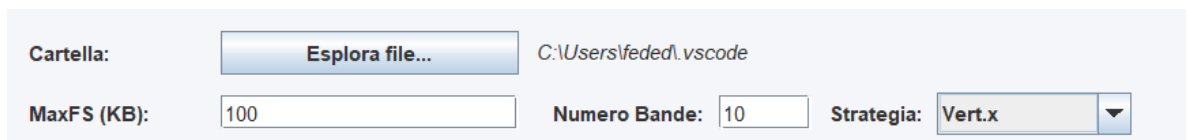
3. ESTENSIONE INTERATTIVA

Come richiesto, è stata creata un'estensione interattiva della libreria che permette di avere una migliore esperienza utente:

- Grafico: rappresenta la distribuzione delle varie fasce come un ortogramma, con aggiornamento in tempo reale quando viene eseguito il report.
- Pulsanti start e stop: il pulsante start inizia la creazione in un report sulla directory selezionata. Durante l'esecuzione sarà possibile cliccare il pulsante stop per ottenere uno snapshot di un determinato momento dell'esplorazione.



- Impostazioni: Prima dell'esecuzione è possibile modificare il numero di bande, la dimensione massima dei file, la directory di partenza e il paradigma utilizzato, eliminando ogni dipendenza dal cambio manuale a livello di codice di questi parametri.



4. CONCLUSIONI

In conclusione, come ci dimostrano i vari snapshot del completamento della stessa ricerca, si possono notare evidenti differenze di performance. Questo non è dovuto al fatto che un modello possa essere “migliore” di un altro, quanto al fatto che in un contesto di molteplici operazioni I/O certi approcci hanno dei trade-off migliori di altri:

- Vert-x: ha risentito della rigidità del modello Event Loop applicato a un'operazione di I/O bloccante e ricorsiva locale su disco. La necessità di delegare il lavoro a thread di worker o di gestire catene complesse di callback asincrone ha introdotto un overhead superiore rispetto agli altri due paradigmi

- Rx Reactive: pur introducendo il leggero costrutto d'astrazione proprio dei Reactive Streams, gestisce la ricerca ricorsiva con un singolo thread in maniera asincrona. Questo garantisce comunque un throughput eccellente grazie all'ottimizzazione degli schedulers specializzati per I/O
- Virtual-Threads: Ottimizzati per operazioni I/O, i virtual Thread si sono rivelati la scelta più performante. Questo risultato conferma come la concorrenza strutturata e imperativa si adatti in modo ottimale alla natura ricorsiva e I/O-bound della scansione di un File System.